Smart Vehicle Impound & Violation Management System

A database design project digitizing government traffic operations — managing vehicle registration, monitoring violations, and coordinating impoundment across designated lots.

DATABASE DESIGN

GOVERNMENT IT



Made with GAMMA

Mini-World Overview

This system replaces paper-based traffic enforcement with a structured digital workflow. It connects five core entities to manage the full lifecycle of a traffic violation — from detection to resolution.

Person (Owner)

owner_id, name, address, age

Vehicle

plate_number, model, color, license status

Impound Record

record_id, fine, payment status, violation type

Police Officer

police_id, name, department

Impound Lot

lot_id, location, capacity

Made with GAMMA

Core Business Rules



One Owner Per Vehicle

Every vehicle must be registered to exactly one person. Ownership is a strict 1:1 relationship from the vehicle's perspective.

Violation Types

A violation can be a simple fine or an **Impoundment** — the latter requires assignment to a valid Impound Lot ID.

Record Closure Rule

A record is marked **Closed** only when `paid_amount = fine`. Partial payments leave the record open.

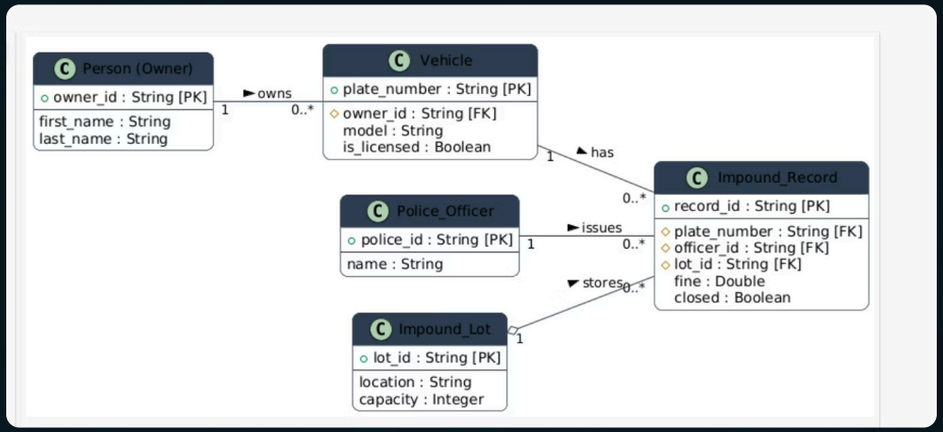
Plate Number Search

Searching by plate number must return both vehicle details *and* the owner's identity via SQL JOIN.

Made with GAMMA

ER Diagram — Crow's Foot Notation

The conceptual model uses Crow's Foot notation to express three key relationships, each with clear cardinality constraints.



Cardinality Summary

1 Person → Vehicle: 1:N

One person can own multiple vehicles. Each vehicle links to exactly one owner via `owner_id` FK.

2 Vehicle → Record: 1:N

A single vehicle may accumulate multiple violation records over time.

3 Lot → Record: 1:N

One impound lot can store many vehicles simultaneously, referenced by `lot_id` FK.

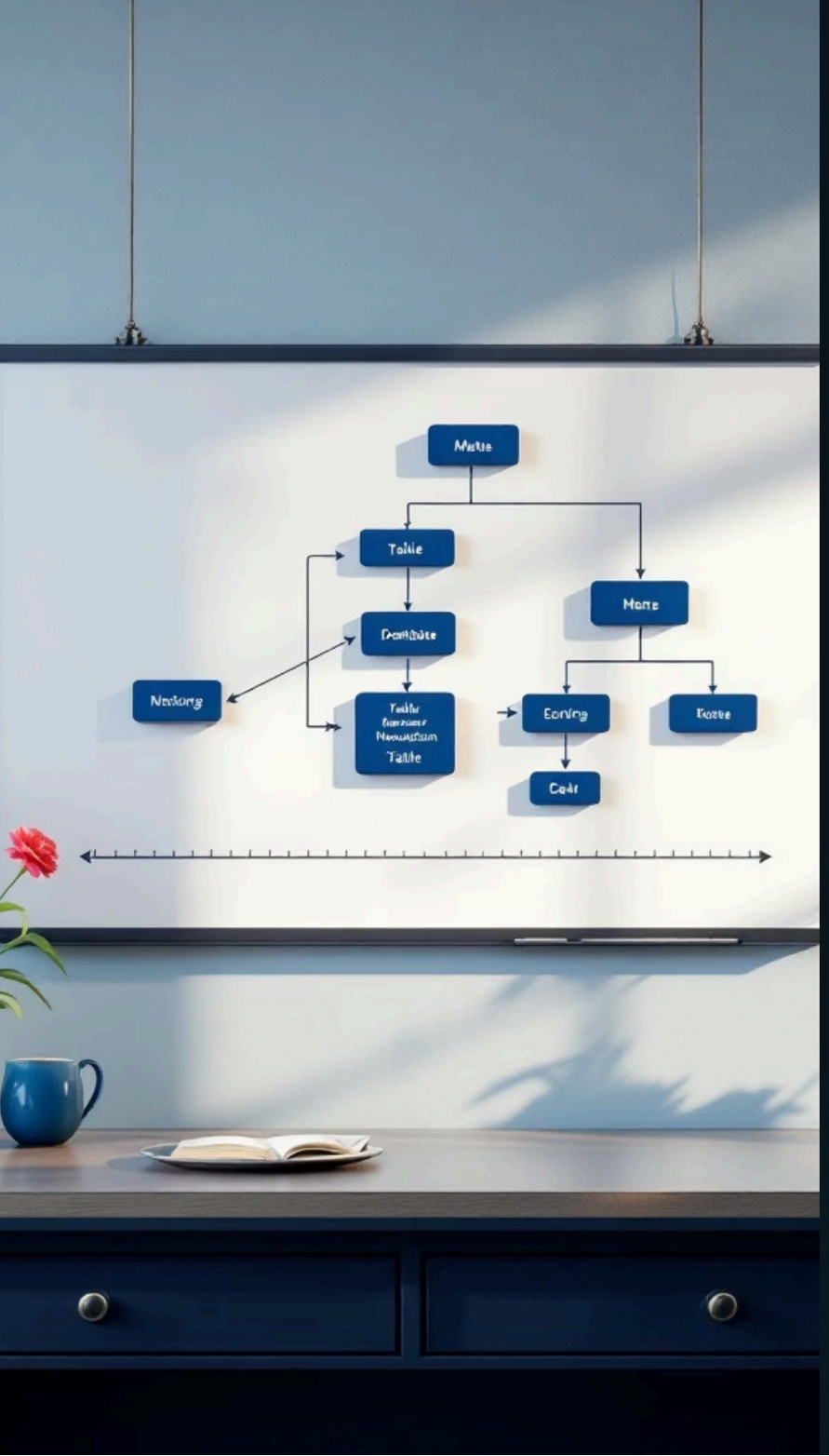
Made with GAMMA

Relational Schema — ER to Tables

The conceptual ER diagram is mapped into five normalized relational tables. Primary keys are underlined; foreign keys are italicized.

Table	Attributes	Key Notes
Persons	owner_id, first_name, last_name, address, age	PK: owner_id
Vehicles	plate_number, color, model, vehicle_type, *owner_id*, is_licensed	PK: plate_number; FK → Persons
Police	police_id, first_name, last_name	PK: police_id
Impound_Lots	lot_id, location, capacity	PK: lot_id
Impound_Records	record_id, record_type, fine, paid_amount, closed, *plate_number*, *officer_id*, *lot_id*	PK: record_id; 3 FKs

Made with GAMMA



Normalization — 1NF Through 3NF

1

1NF — Atomic Values

All attributes are atomic with no repeating groups. Multi-value fields (e.g., images) are split into separate columns.

2

2NF — Full Dependency

All non-key attributes fully depend on the Primary Key. Example: `model` depends on `plate_number`, not on `owner_id`.

3

3NF — No Transitivity

Transitive dependencies removed. Owner's address lives in **Persons**, not **Vehicles** — linked via `owner_id` FK.

Made with GAMMA

SQL DDL — Core Table Definitions

The three central tables with primary keys, data types, and foreign key constraints enforcing referential integrity.

Persons & Vehicles

```
CREATE TABLE persons (
  owner_id    TEXT PRIMARY KEY,
  first_name  TEXT NOT NULL,
  last_name   TEXT NOT NULL,
  address     TEXT,
  age         INTEGER
);

CREATE TABLE vehicles (
  plate_number TEXT PRIMARY KEY,
  color        TEXT,
  model        TEXT,
  owner_id     TEXT,
  is_licensed  INTEGER DEFAULT 1,
  FOREIGN KEY (owner_id)
    REFERENCES persons(owner_id)
);
```

Impound_Records

```
CREATE TABLE impound_records (
  record_id   TEXT PRIMARY KEY,
  fine        REAL    NOT NULL,
  paid_amount REAL    DEFAULT 0,
  closed      INTEGER DEFAULT 0,
  plate_number TEXT,
  lot_id      TEXT,
  FOREIGN KEY (plate_number)
    REFERENCES vehicles(plate_number),
  FOREIGN KEY (lot_id)
    REFERENCES impound_lots(lot_id)
);
```

`closed = 0` → Open | `closed = 1` → Closed

Made with GAMMA

Sample Data & Test Case

Insert Statements

```
INSERT INTO persons
VALUES ('O101', 'John', 'Doe',
        'Amman, 7th Circle', 34);

INSERT INTO persons
VALUES ('O102', 'Sami', 'Ahmad',
        'Irbid, City Center', 28);

INSERT INTO vehicles
VALUES ('AB-123', 'White',
        'Toyota Camry', 'O101', 1);

INSERT INTO impound_records
VALUES ('REC-001', 50.0, 0.0,
        0, 'AB-123', NULL);
```

Test Case — Plate Number Search

Verifies the JOIN between **Vehicles** and **Persons** returns owner identity alongside vehicle data.

```
SELECT v.plate_number, v.model,
       p.first_name, p.last_name
FROM   vehicles v
JOIN   persons p
  ON   v.owner_id = p.owner_id
WHERE  v.plate_number = 'AB-123';
```

Expected output: **AB-123**, Toyota Camry, **John Doe**

Made with GAMMA

Challenges & Insights

Key Challenge — Viewer Search

Implementing plate-number search required careful SQL JOIN logic to retrieve the owner's name correctly — not just vehicle data. Ensuring the JOIN condition matched the correct foreign key was critical to avoid null results.

Architectural Insight

Separating **DatabaseManager** (data access logic) from **ApiController** (web controller) significantly improved code clarity and maintainability — a clean separation of concerns.



Made with GAMMA

Future Improvements & Team

Next Version Roadmap

01

Push Notification System

Alert vehicle owners via email the moment a violation is recorded — reducing resolution time.

02

Partial Payment Tracking

Extend the payment model to support installment plans with a payment history sub-table.

03

Lot Capacity Alerts

Trigger warnings when an impound lot nears full capacity, enabling proactive coordination.

Project Details

Group Members

- Hamza
- Mohammed
- Juana

Course: Database Design

Domain: Government Traffic Enforcement

Made with GAMMA